

DOCUMENTO 4: ROADMAP 10 SEMANAS



MAPEO A PARCIALES DEL CURSO

PARCIAL 1 (Sem 1-3): Diseño de Arquitectura + Microservicios

PARCIAL 2 (Sem 4-7): Desarrollo Frontend + Backend

PARCIAL 3 (Sem 8-10): Testing + SonarQube + Presentación



SEMANA 1-3: PARCIAL 1 (ARQUITECTURA + MVP CORE)

SEMANA 1: Setup AWS + Cognito + GitHub

Responsables: P3 (Lead), P1 (Frontend), P2 (Backend)

Tareas Principales:

P3 (DevOps):

- ✓ AWS account (estudiante) + billing alerts
- ✓ VPC + Subnet (pública) + Security Groups
- ✓ IAM básico (si disponible) o Security Groups
- ✓ GitHub repo setup + branch protection rules
- ✓ CI/CD skeleton (GitHub Actions)
- ✓ CloudWatch logging enabled
- ✓ Presupuesto tracking sheet

P2 (Backend):

- ✓ Cognito User Pool creado
- ✓ JWT configuration (15 min access, 30d refresh)
- ✓ Cognito public key exported (para Lambda)
- ✓ Local Python environment (3.11)
- ✓ Pytest + Pydantic setup

P1 (Frontend):

- ✓ React app creado (Vite)
- ✓ TypeScript configuration

- ✓ Tailwind CSS setup
- ✓ Local development server
- ✓ Cognito SDK (@aws-amplify) installed

Learning Objectives:

- Entender VPC, Subnets, Security Groups
- Cómo funciona Cognito User Pool
- GitHub Actions basics
- Local development setup

Criterios de Done:

- ☐ AWS account funcional con billing alerts
- ☐ GitHub repo con branch protection
- ☐ P1 puede login en React local
- ☐ P2 puede crear Lambda local
- ☐ P3 puede deployer a Lambda manual

Hito Semanal:

- P1, P2, P3 conectados a Cognito
- Pueden hacer signup/login desde React
- Cognito JWT se imprime en console

SEMANA 2: Lambda 1 (Usuarios) + API Gateway

Responsables: P2 (Lead), P3 (AWS), P1 (Frontend)

Tareas Principales:

P2 (Backend) :

- ✓ Lambda function Usuarios (Python)
 - POST /auth/signup
 - POST /auth/login
 - POST /auth/refresh
- ✓ Integration con Cognito SDK
- ✓ JWT token validation
- ✓ Unit tests (Pytest) > 80% coverage
- ✓ Local testing con sam local

P3 (DevOps) :

- ✓ API Gateway creado
- ✓ Lambda permissions (ejecución)

- ✓ Cognito authorizer integrado
- ✓ Rate limiting (1000 req/min)
- ✓ CloudWatch logs configured
- ✓ GitHub Actions: deploy Lambda

P1 (Frontend) :

- ✓ Login form connected to API Gateway
- ✓ Error handling (401, 403)
- ✓ JWT stored en IndexedDB
- ✓ E2E test con Playwright
- ✓ Offline fallback (localización JWT)

Learning Objectives:

- Cómo funciona Lambda (cold start, warm)
- API Gateway routing + auth
- Testing Lambda functions
- JWT management en frontend

Criterios de Done:

- ☐ Lambda Usuarios responde correctamente
- ☐ JWT se valida en API Gateway
- ☐ P1 puede signup/login desde PWA
- ☐ Cobertura > 80% en Pytest
- ☐ SonarQube pasa análisis

Hito Semanal:

- Usuario se puede registrar y logear
- Token JWT funcionando end-to-end
- API Gateway routea correctamente

SEMANA 3: Lambda 2 (Incidencias) + DynamoDB + Frontend

Responsables: P2 (Lead), P3 (DynamoDB), P1 (UI)

Tareas Principales:

P2 (Backend) :

- ✓ Lambda function Incidencias (Python)
 - POST /reports (create)
 - GET /reports (list)
- ✓ DynamoDB put_item + query

- ✓ Validación Pydantic (estructura JSON)
- ✓ Error handling
- ✓ Unit tests > 80%

P3 (DevOps):

- ✓ DynamoDB tabla "reportes" creada
- ✓ Partition key: report_id
- ✓ GSI: location (para queries geográficas)
- ✓ TTL: 90 días (auto-delete)
- ✓ Streams enabled (para siguiente semana)
- ✓ GitHub Actions: test + deploy

P1 (Frontend):

- ✓ Reporte form (foto + ubicación + detalles)
- ✓ Llamada a Lambda Incidencias
- ✓ Validación de entrada
- ✓ Error states
- ✓ Offline: guardado en IndexedDB + Service Worker
- ✓ Sync cuando recupera conexión
- ✓ E2E tests

Learning Objectives:

- DynamoDB design (partition key, GSI)
- Stream y eventos (preparación)
- Offline-first architecture
- Service Workers basics

Criterios de Done:

- ☐ DynamoDB tabla funcional
- ☐ Lambda Incidencias crea reportes
- ☐ Frontend envía reportes correctamente
- ☐ Offline mode funciona (IndexedDB)
- ☐ SonarQube > 80% coverage

Hito Semanal:

- Usuario puede reportar incendio
- Reporte se guarda en DynamoDB
- Si está offline: se sincroniza después
- Datos persisten

ENTREGABLE PARCIAL 1:

- Documento: Arquitectura de 4 microservicios
 - Diagrama: Flujos principales
 - Justificación: por qué cada decisión
 - Código: Lambda 1-2 con tests
 - Presentación: 10 min a docente
-

SEMANA 4-7: PARCIAL 2 (DESARROLLO COMPLETO)

SEMANA 4: EC2 FASTapi + Rekognition

Responsables: P2 (Lead), P3 (Infra), P1 (UI)

Tareas Principales:

P2 (Backend):

- ✓ FASTapi app (EC2 local machine o Docker)
 - POST /validate (recibe s3_image_url)
 - Llama Rekognition API
 - Almacena resultado en DynamoDB
- ✓ Docker image Alpine Python (< 100MB)
- ✓ Dockerfile + docker-compose
- ✓ Unit tests (Pytest)
- ✓ Error handling (Rekognition fails)

P3 (DevOps):

- ✓ EC2 security group (puerto 443, 22)
- ✓ EC2 subnet configurada (privada o pública)
- ✓ Docker registrado en EC2
- ✓ Deployment script (GitHub Actions → EC2)
- ✓ CloudWatch agent (logs desde EC2)
- ✓ Auto-shutdown script (si no se usa)

P1 (Frontend):

- ✓ S3 pre-signed URL upload
- ✓ Imagen sube directo a S3 (no pasa por Lambda)
- ✓ Llama validación (POST /validate + s3_url)
- ✓ Muestra resultado: "Incendio validado ✓"
- ✓ UI para diferentes resultados

Learning Objectives:

- Docker + Alpine (tamaño mínimo)
- EC2 operations + security groups
- API calls a servicios AWS externos
- CI/CD deploy a EC2

Criterios de Done:

- ☐ Imagen sube a S3 correctamente
- ☐ EC2 FASTapi recibe validación
- ☐ Rekognition analiza (5k/mes free)
- ☐ Resultado en DynamoDB
- ☐ Docker image < 100MB
- ☐ Tests > 80%

Hito Semanal:

- Imagen se sube, se valida, se guarda
- EC2 responde correctamente
- Flujo end-to-end funciona

SEMANA 5: Lambda 3 (Notificaciones) + Streams

Responsables: P2 (Lead), P3 (SNS/SES), P1 (Notifications UI)

Tareas Principales:

P2 (Backend):

- ✓ Lambda Notificaciones (Python)
 - Trigger: DynamoDB Streams (INSERT en reportes)
 - Calcula radio 5km (geospatial query)
 - Query usuarios en zona
- ✓ Integración con SNS + SES
- ✓ Error handling + retries
- ✓ Unit tests > 80%
- ✓ Local testing

P3 (DevOps):

- ✓ DynamoDB Streams: modo NEW_AND_OLD_IMAGES
- ✓ Lambda trigger (Event Source Mapping)
- ✓ SNS topic creado + subscriptions
- ✓ SES sender identity verified
- ✓ IAM role para Lambda (SNS + SES publish)
- ✓ GitHub Actions: deploy Lambda

P1 (Frontend) :

- ✓ Permiso de notificaciones (browsers)
- ✓ Service Worker: escucha push notifications
- ✓ UI: muestra alertas recibidas
- ✓ Historial de notificaciones
- ✓ Offline: almacena msgs en IndexedDB

Learning Objectives:

- DynamoDB Streams (event-driven)
- SNS for push (fan-out)
- SES for email (transactional)
- Geospatial queries (básico)

Criterios de Done:

- ☐ Reporte trigger Lambda Notificaciones automático
- ☐ SNS notificación llega (mock si no hay device real)
- ☐ SES email enviado
- ☐ Usuarios cercanos notificados
- ☐ Tests > 80%

Hito Semanal:

- Se crea reporte → dispara notificaciones automático
 - Usuarios en zona reciben alerta
 - Email se envía
-

SEMANA 6: Sync DynamoDB → PostgreSQL

Responsables: P3 (Lead), P2 (Lambda Processor)

Tareas Principales:

P3 (DevOps) :

- ✓ PostgreSQL en EC2 (local)
- ✓ Schema: reportes_agg, metrics_hourly
- ✓ Connection string configurado
- ✓ Backups automáticos

P2 (Backend) :

- ✓ Lambda Processor (Python)
 - Trigger: CloudWatch Events (cada 5 min)

- Lee CloudWatch Logs (fuente de verdad)
 - Agrega datos
 - INSERT a PostgreSQL
- ✓ Error handling + retries
- ✓ Tests > 80%

P1 (Frontend):

- ✓ (Puede esperar, es para Grafana)

Learning Objectives:

- Event-driven architecture
- Data aggregation (log streams)
- PostgreSQL basics
- AWS Lambda scheduling (CloudWatch Events)

Criterios de Done:

- ☐ Lambda Processor ejecuta cada 5 min
- ☐ Logs se leen correctamente
- ☐ PostgreSQL se actualiza
- ☐ Sin duplicados
- ☐ Tests > 80%

Hito Semanal:

- PostgreSQL tiene datos en tiempo real
- Listo para Grafana

SEMANA 7: Grafana + 2 Perfiles + Testing Integración

Responsables: P3 (Grafana), P1 + P2 (Integration tests)

Tareas Principales:

P3 (DevOps):

- ✓ Grafana en EC2 (:3000)
- ✓ DataSource: PostgreSQL
- ✓ Dashboard TI (privado, localhost)
 - Todos los logs
 - Performance metrics
 - Error rates
- ✓ Dashboard Emergencias (público vía Cognito)
 - Reportes validados

- Mapa (Leaflet + reportes geoloc)
- Alertas activas
- ✓ Auth: Cognito OIDC plugin

P1 + P2 (Testing):

- ✓ Integration tests (end-to-end)
 - Crear usuario → reportar → validar → notificar
- ✓ E2E con Playwright
- ✓ Load test básico (50 reportes simul)
- ✓ Coverage mínimo 60% (SonarQube)

Learning Objectives:

- Monitoring en producción (Grafana)
- 2 perfiles de usuarios (RBAC básico)
- Integration testing
- Load testing

Criterios de Done:

- ☐ Grafana TI accesible (localhost)
- ☐ Grafana Emergencias accesible (Cognito)
- ☐ Dashboards muestran datos
- ☐ E2E test: crear reporte → notificación
- ☐ Coverage > 60%

Hito Semanal:

- Sistema completo funcionando
- Todos los datos visibles en Grafana
- Listo para Parcial 2

ENTREGABLE PARCIAL 2:

- Sistema funcional 100%
- 4 microservicios running
- Frontend + Backend integrados
- Tests > 60%
- Presentación: demostración en vivo

SEMANA 8-10: PARCIAL 3 (TESTING + SONARQUBE + PRESENTACIÓN)

SEMANA 8: SonarQube + Code Quality

Responsables: P3 (Lead), P2 (Code review), P1 (Code review)

Tareas Principales:

P3 (DevOps) :

- ✓ SonarQube Community instalado
- ✓ GitHub Actions: ejecuta análisis en cada push
- ✓ Quality gate: > 60% coverage
- ✓ Refactor code según reportes
- ✓ Code smells solucionados

P2 (Backend) :

- ✓ Unit tests: todos los Lambda functions
- ✓ Coverage: apunta a > 80%
- ✓ Refactor: mejorar SOLID principles
- ✓ Security: revisar inputs (SQL injection, etc)
- ✓ Performance: optimize queries

P1 (Frontend) :

- ✓ Unit tests: componentes React
- ✓ Coverage: > 70%
- ✓ E2E: todos los flujos críticos
- ✓ Accessibility: WCAG 2.1 AA (básico)
- ✓ Performance: lighthouse > 80

Learning Objectives:

- Code quality metrics
- Test coverage
- Static analysis (SonarQube)
- Refactoring

Criterios de Done:

- ☐ SonarQube analysis: 0 blocker issues
- ☐ Coverage > 60% (backend), > 70% (frontend)
- ☐ All tests passing
- ☐ Lighthouse > 80
- ☐ No security vulnerabilities

Hito Semanal:

- Código limpio + bien testeado
 - SonarQube aprobado
-

SEMANA 9: Load Testing + Optimización

Responsables: P3 (Lead), P2 (Performance)

Tareas Principales:

P3 (DevOps):

- ✓ Load test: 100 reportes simultáneos
- ✓ Apache JMeter scripts
- ✓ Monitorear CloudWatch metrics
- ✓ Identificar cuellos de botella
- ✓ Auto-scaling validation (Lambda)

P2 (Backend):

- ✓ Optimize Lambda performance
- ✓ Reduce cold starts (provisioned concurrency NO, cost)
- ✓ DynamoDB: verify indexes
- ✓ Rekognition: parallelizar si posible

P1 (Frontend):

- ✓ Bundle size optimization
- ✓ Lazy loading (code splitting)
- ✓ Service Worker caching strategies

Learning Objectives:

- Load testing(JMeter)
- Performance optimization
- CloudWatch metrics
- Auto-scaling understanding

Criterios de Done:

- ☐ 100 req/sec handled without errors
- ☐ Latency < 500ms(P95)
- ☐ Lambda scales automatically
- ☐ Zero cost overruns
- ☐ Bottlenecks identified + solved

Hito Semanal:

- Sistema escala a 100+ eventos/sec
 - Performance validado
-

SEMANA 10: Documentación Final + Presentación

Responsables: Todos (P1, P2, P3)

Tareas Principales:

P3 (Lead):

- ✓ Architecture documentation (final)
- ✓ Deployment guide (how to reproduce)
- ✓ Cost breakdown (what we spent)
- ✓ Lessons learned (what was hard)
- ✓ Future improvements

P2 (Backend):

- ✓ API documentation (endpoint descriptions)
- ✓ Database schema (DynamoDB + PostgreSQL)
- ✓ Microservices guide
- ✓ Testing guide (how to run Pytest)

P1 (Frontend):

- ✓ Component documentation
- ✓ Setup guide (how to run React local)
- ✓ Offline mode guide
- ✓ Screenshots / demo

ALL (Presentation):

- ✓ Slides: 20-30 min
 - Problem statement
 - Architecture decisions
 - Tech stack justification
 - Live demo
 - Challenges faced
 - Lessons learned
 - What would you do different
- ✓ Live demo: create report → validate → notify
- ✓ Q&A prep (likely questions)

Learning Objectives:

- Documentation best practices
- Presentation skills
- Reflecting on design decisions
- Teaching others

Criterios de Done:

- ☐ All code documented
- ☐ README complete
- ☐ Deployment guide works
- ☐ Slides prepared
- ☐ Live demo works without errors
- ☐ Team can answer any question

Hito Semanal:

- Presentación lista
- Docente entiende arquitectura
- Equipo defiende decisiones

ENTREGABLE FINAL:

- Código en GitHub (clean, well-tested)
- SonarQube: > 60% coverage
- Documentación completa
- Presentación 20-30 min
- Live demo funcionando



DISTRIBUCIÓN DE ESFUERZO (por semana)

Semana	P1 Frontend	P2 Backend	P3 DevOps	TOTAL
1	30%	30%	40%	100%
2	40%	40%	20%	100%
3	40%	35%	25%	100%
4	30%	40%	30%	100%
5	35%	35%	30%	100%
6	20%	30%	50%	100%
7	50%	25%	25%	100%
8	40%	40%	20%	100%

Semana	P1 Frontend	P2 Backend	P3 DevOps	TOTAL
9	30%	50%	20%	100%
10	33%	33%	34%	100%

Notas:

- Semana 1-3: P3 carga más (infraestructura)
- Semana 4-7: Todos equilibrado (desarrollo)
- Semana 8-10: Todos colaboran (finales)

CHECKLIST FINAL (Antes de Presentación)

CÓDIGO:

- [] GitHub repo limpio
- [] .gitignore correcto
- [] README actualizado
- [] Documentación en /docs
- [] All branches merged to main

TESTING:

- [] Pytest: todos pasan
- [] Playwright: E2E pass
- [] Coverage > 60%
- [] SonarQube: aprobado
- [] Load test: 100 req/sec OK

AWS:

- [] Lambda functions deployed
- [] API Gateway working
- [] DynamoDB populated
- [] EC2 FASTapi running
- [] Cognito configured
- [] Grafana 2 profiles active
- [] Cost tracking: < \$10

PRESENTACIÓN:

- [] Slides ready (20-30 min)
- [] Live demo tested 3x
- [] Team answers prepared

- [] Architecture diagram ready
 - [] Cost breakdown shown
-

DEFENSA: PREGUNTAS ESPERADAS DEL DOCENTE

1. "¿Por qué eligieron 4 microservicios?"
→ Respuesta: Separación de concerns, escalabilidad independiente
 2. "¿Por qué DynamoDB y no PostgreSQL?"
→ Respuesta: Free tier permanente, escalabilidad, event-driven (Streams)
 3. "¿Cómo manejaron offline?"
→ Respuesta: Service Workers + IndexedDB + sync automático
 4. "¿Cuánto costó realmente?"
→ Respuesta: ~\$3.50 (presupuesto \$50, margen 93%)
 5. "¿Qué harían diferente?"
→ Respuesta: [Tu reflexión honesta]
 6. "¿Puede escalar a 1M usuarios?"
→ Respuesta: Sí, Lambda + DynamoDB es on-demand. RDS no.
-

Listo para 10 semanas de ejecución. 🚀